BỘ GIÁO DỤC VÀ ĐÀO TẠO - ĐẠI HỌC THÁI NGUYÊN
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG (ICTU)
KHOA CÔNG NGHỆ THÔNG TIN

BÁO CÁO TỔNG HỢP KẾT QUẢ ĐỀ TÀI

HỌC PHẦN: TRIỂN KHAI VÀ QUẢN TRỊ HỆ THỐNG PHẦN MỀM

CHỦ ĐỀ ĐỀ TÀI:

HỆ THỐNG QUẢN LÝ THƯ VIỆN

Giảng viên hướng dẫn: TS. Nguyễn Anh Chuyên
Sinh viên thực hiện: Phạm Tiến Toàn
Mã số sinh viên: DTC245200032
Lớp: CNTT K23A

THÁI NGUYÊN, NĂM 2026

LỜI CAM ĐOAN

Tôi xin cam đoan rằng toàn bộ nội dung trong bản báo cáo tổng hợp với đề tài **"Hệ thống Quản lý Thư viện trên nền tảng Docker Compose, Nginx Reverse Proxy và Hệ thống Giám sát Tập trung"** là công trình do tôi tự nghiên cứu, thiết kế, hiện thực mã nguồn và triển khai kiểm thử dưới sự hướng dẫn chuyên môn trực tiếp của giảng viên hướng dẫn.

Tôi xin khẳng định các cam kết sau:
1. Toàn bộ mã nguồn triển khai dự án (FastAPI, Nginx, PostgreSQL, Docker Compose, Prometheus, Grafana, Promtail, Loki) và số liệu thực nghiệm được hình thành trung thực từ quá trình làm việc thực tế trên môi trường máy chủ cục bộ.
2. Các thông tin trích dẫn, mô hình công nghệ chuẩn và công cụ mã nguồn mở đều được ghi nhận nguồn gốc rõ ràng, tuân thủ đạo đức học thuật và các quy định của Khoa Công nghệ Thông tin - Trường Đại học Công nghệ Thông tin và Truyền thông.
3. Báo cáo không có sự sao chép bất hợp pháp từ bất kỳ đồ án hoặc khóa luận nào khác.

Thái Nguyên, ngày ....tháng ....năm 2026
Sinh viên thực hiện đề tài

Phạm Tiến Toàn
(Mã SV: DTC245200032)

LỜI CẢM ƠN

Để hoàn thành đồ án học phần một cách toàn diện và đạt tiêu chuẩn kỹ thuật cao, em xin bày tỏ lòng biết ơn chân thành nhất đến Ban Giám hiệu **Trường Đại học Công nghệ Thông tin và Truyền thông (ICTU)** cùng quý Thầy/Cô giáo thuộc **Khoa Công nghệ Thông tin**, những người đã truyền dạy tri thức nền tảng vững vàng về kỹ thuật lập trình mạng, cơ sở dữ liệu và kiến trúc hệ thống phân tán.

Đặc biệt, em xin gửi lời cảm ơn sâu sắc tới **TS. Trần Quang Quý** - người đã tận tâm giảng dạy, định hướng kiến trúc chuẩn DevOps hiện đại, giải thích cặn kẽ các bài toán cô lập mạng, cơ chế cân bằng tải, bảo mật SSL và giám sát log tập trung. Những lời chỉ dẫn

quý báu của Thầy là kim chỉ nam giúp em giải quyết các bài toán kỹ thuật phức tạp trong toàn bộ các bước triển khai của đồ án này.

MỤC LỤC TỔNG QUAN HỆ THỐNG

(Mục lục tự động được liên kết trực tiếp với các tiêu đề mục. Bạn có thể nhấn chuột phải chọn 'Update Field' hoặc phím F9 trong Word để cập nhật lại số trang bất kỳ lúc nào)

LỜI CAM ĐOAN 2

LỜI CẢM ƠN 2

MỤC LỤC TỔNG QUAN HỆ THỐNG 3

DANH MỤC TỪ VIẾT TẮT VÀ THUẬT NGỮ KỸ THUẬT.....5

DANH MỤC HÌNH ẢNH MINH HỌA.....6

PHẦN 1: TỔNG QUAN VỀ CHỦ ĐỀ.....7

1.1. Giới thiệu tổng quan.....7

1.2. Các khái niệm kỹ thuật cốt lõi 7

1.3. Lý do chủ đề có tính cấp thiết và quan trọng.....8

PHẦN 2: PHÂN TÍCH CHUYÊN SÂU HỆ THỐNG 9

2.1. Kiến trúc phân tầng và phân bổ các dịch vụ (Services Topology) 9

2.2. Bảng tổng hợp thông số kỹ thuật đầy đủ các dịch vụ trong hệ thống.....10

2.3. Cơ chế cô lập mạng an toàn và nguyên tắc Least Privilege (Least Exposure) 11

PHẦN 3: HƯỚNG DẪN TỪNG BƯỚC TRIỂN KHAI VÀ VẬN HÀNH HỆ THỐNG 14

3.1. BƯỚC 1: Xây dựng Ứng dụng Quản lý Thư viện (FastAPI, Jinja2, ORM) 14

3.2. BƯỚC 2: Cấu hình Container hóa PostgreSQL, pgAdmin & Phân quyền 15

3.3. BƯỚC 3: Thiết lập Nginx Reverse Proxy, SSL HTTPS & Security Headers 16

3.4. BƯỚC 4: Vận hành, Tự động hóa Khởi động & Kiểm thử Toàn diện.....16

3.5. BƯỚC 5: Thiết lập Hệ thống Thu thập & Trực quan hóa Metrics (Prometheus & Grafana) 16

3.6. BƯỚC 6: Xây dựng Hệ thống Quản lý Nhật ký Tập trung (Loki & Promtail).....18

PHẦN 4: KẾT LUẬN VÀ KHUYẾN NGHỊ THỰC TIỄN.....20

4.1. Tóm tắt kết quả đạt được 20

4.2. Khuyến nghị và lưu ý kỹ thuật khi áp dụng vào thực tế 20

TÀI LIỆU THAM KHẢO.....21

PHỤ LỤC: MÃ NGUỒN CẤU HÌNH HỆ THỐNG VÀ TRUY VẤN MẪU 21
Phụ lục A: Định nghĩa Dịch vụ Docker Compose (Trích đoạn cốt lõi).....21
Phụ lục B: Cấu trúc thư mục nguồn dự án 22

DANH MỤC TỪ VIẾT TẮT VÀ THUẬT NGỮ KỸ THUẬT

Từ viết tắt	Thuật ngữ Tiếng Anh	Ý nghĩa / Ứng dụng kỹ thuật trong đồ án
API	Application Programming Interface	Giao diện lập trình ứng dụng RESTful cung cấp dịch vụ quản lý thư viện
ASGI	Asynchronous Server Gateway Interface	Giao diện cổng máy chủ bất đồng bộ giúp Uvicorn/FastAPI đạt hiệu năng cao
CSP	Content Security Policy	Chính sách bảo mật nội dung HTTP Header ngăn chặn XSS và chèn mã độc
DBMS	Database Management System	Hệ quản trị cơ sở dữ liệu quan hệ (PostgreSQL 16) lưu trữ thông tin nghiệp vụ
EOL	End Of Life	Giai đoạn kết thúc vòng đời hỗ trợ kỹ thuật (Lưu ý với Promtail v3.0.0 từ tháng 3/2026)
HTTPS	Hypertext Transfer Protocol Secure	Giao thức truyền siêu văn bản an toàn mã hóa qua SSL/TLS (Cổng 443)
IaC	Infrastructure as Code	Mô hình quản lý hạ tầng bằng mã nguồn thông qua tệp cấu hình docker-compose.yml
LogQL	Log Query Language	Ngôn ngữ truy vấn nhật ký của Grafana Loki lấy cảm hứng từ PromQL

ORM	Object-Relational Mapping	Kỹ thuật ánh xạ bảng CSDL thành các đối tượng Python thông qua SQLAlchemy
PromQL	Prometheus Query Language	Ngôn ngữ truy vấn mạnh mẽ tính toán các chỉ số metrics chuỗi thời gian
RDBMS	Relational Database Management System	Hệ thống quản trị cơ sở dữ liệu có quan hệ toàn vẹn khóa ngoại
SSL/TLS	Secure Sockets Layer / Transport Layer Security	Tầng bảo mật truyền tải mã hóa dữ liệu đường truyền máy khách - máy chủ
UI/UX	User Interface / User Experience	Giao diện và trải nghiệm người dùng trên ứng dụng Web thư viện
XSS	Cross-Site Scripting	Lỗ hổng bảo mật tấn công chèn mã kịch bản độc hại phía người dùng

DANH MỤC HÌNH ẢNH MINH HỌA

Hình 2.1: Sơ đồ kiến trúc phân tầng tổng thể của Hệ thống Quản lý Thư viện trên Docker Compose

Hình 2.2: Cơ chế cô lập mạng riêng ảo (Bridge Network) và bảo vệ cơ sở dữ liệu nội bộ

Hình 3.1: Quy trình kỹ thuật 6 bước triển khai toàn diện Hệ thống Quản trị Phần mềm

Hình 3.2: Giao diện bảng điều khiển trực quan (Dashboard) của Website Quản lý Thư viện

Hình 3.3: Bảng điều khiển giám sát hiệu năng thời gian thực trên Grafana Dashboard

Hình 3.4: Giao diện tra cứu nhật ký tập trung trên Grafana Explore thông qua cú pháp LogQL

PHẦN 1: TỔNG QUAN VỀ CHỦ ĐỀ

1.1. Giới thiệu tổng quan

Trong bối cảnh chuyển đổi số giáo dục đại học, các thư viện truyền thống đang dần chuyển dịch sang mô hình trung tâm thông tin số hóa. Việc triển khai các ứng dụng phần mềm đơn lẻ (Desktop Apps) hoặc ứng dụng web nguyên khối (Monolith) cài đặt thủ công trực tiếp lên hệ điều hành máy chủ thường bộc lộ nhiều điểm hạn chế: phụ thuộc sâu vào môi trường máy tính, khó nâng cấp các gói thư viện và tiềm ẩn nguy cơ bảo mật nghiêm trọng khi cơ sở dữ liệu bị để lộ cổng kết nối ra ngoài mạng.

Đề tài tập trung vào việc nghiên cứu, hiện thực và vận hành một **Hệ thống Quản lý Thư viện đa dịch vụ (Multi-services Architecture)** hoàn chỉnh trên nền tảng Docker Compose. Toàn bộ các dịch vụ từ cổng giao tiếp biên (Nginx Reverse Proxy), ứng dụng web nghiệp vụ (FastAPI), hệ quản trị cơ sở dữ liệu (PostgreSQL), giao diện quản trị dữ liệu (pgAdmin) cho tới hệ thống giám sát hiệu năng (Prometheus, Grafana) và quản lý nhật ký tập trung (Loki, Promtail) đều được đóng gói, cấu hình mạng nội bộ cô lập và vận hành đồng bộ.

1.2. Các khái niệm kỹ thuật cốt lõi

1. Containerization (Container hóa): Công nghệ ảo hóa ở cấp độ hệ điều hành (OS-level virtualization), cho phép đóng gói ứng dụng cùng mã nguồn, môi trường thực thi và tất cả thư viện phụ thuộc vào một vùng chứa (container) độc lập, bảo đảm tính nhất quán trên mọi môi trường triển khai.
2. Reverse Proxy và SSL Termination: Cơ chế đặt một máy chủ biên (Nginx) đứng trước ứng dụng nội bộ. Nginx tiếp nhận các kết nối mã hóa HTTPS (cổng 443) từ máy khách, thực hiện giải mã SSL (SSL Termination), gắn thêm các tiêu đề bảo mật HTTP Security Headers rồi chuyển tiếp yêu cầu HTTP nguyên bản vào mạng nội bộ.
3. Mạng nội bộ cô lập (Isolated Bridge Network): Mạng ảo do Docker khởi tạo riêng cho hệ thống (gồm 3 mạng riêng biệt: frontend_network (tiếp nhận request), backend_network (cô lập CSDL) và monitoring_network (giám sát metrics & logs)). Trong mạng này, các container phân giải địa chỉ của nhau thông qua dịch vụ DNS nội bộ của Docker theo tên dịch vụ (ví dụ: http://app:8000 hoặc postgres:5432). Cổng CSDL được giấu kín hoàn toàn, không xuất hiện trên bảng định tuyến của máy tính Host.
4. Nguyên tắc phân quyền tối thiểu (Least Privilege): Hạn chế tối đa quyền hạn của người dùng hoặc tiến trình kết nối. Ứng dụng Backend không được phép sử dụng tài khoản quản trị tối cao (superuser) của CSDL mà chỉ được cấp tài khoản giới hạn quyền với các thao tác đọc ghi dữ liệu (DML) thông thường.
5. Hệ thống quan trắc toàn diện (Observability - Metrics & Logs): Khả năng nắm bắt

trạng thái nội tại của hệ thống từ dữ liệu đầu ra bên ngoài thông qua hai trụ cột: Metrics (chuỗi thời gian định lượng độ trễ, tải trọng) và Logs (nhật ký sự kiện chi tiết của từng container).

1.3. Lý do chủ đề có tính cấp thiết và quan trọng

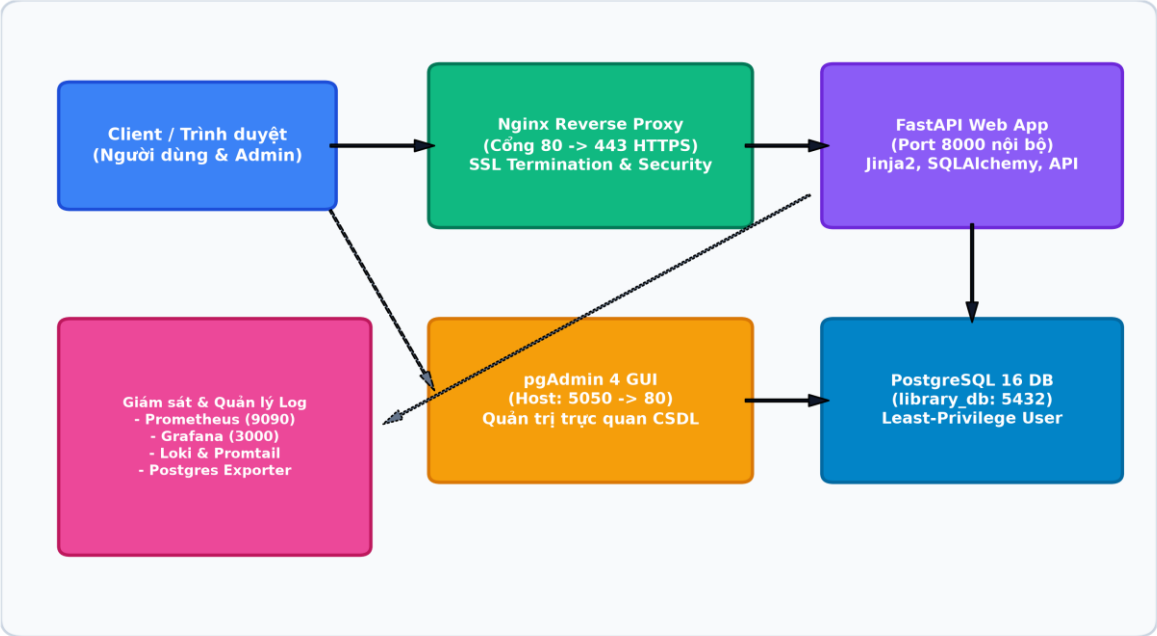
• **Bảo vệ an toàn cơ sở dữ liệu:** Cơ sở dữ liệu thư viện lưu trữ thông tin độc giả, dữ liệu mượn trả và danh mục tài sản. Việc đóng gói và cô lập CSDL trong mạng ảo giúp loại bỏ nguy cơ bị quét cổng tự động, tấn công Brute-force mật khẩu hoặc tấn công khai thác dịch vụ từ mạng bên ngoài.
• **Khả năng vận hành ổn định và độc lập:** Mỗi thành phần chạy trong một không gian riêng. Khi dịch vụ phụ trợ như pgAdmin hoặc Grafana gặp lỗi tải cao, ứng dụng chính và CSDL vẫn tiếp tục phục vụ người dùng bình thường.
• **Tự động hóa triển khai (Infrastructure as Code):** Toàn bộ quy trình cài đặt, liên kết mạng và cấu hình lưu trữ được văn bản hóa thành tệp cấu hình docker-compose.yml. Người quản trị có thể khởi động, sao lưu hoặc di dời toàn bộ hệ thống sang một máy chủ mới trong vài phút mà không cần cài đặt môi trường thủ công.

PHẦN 2: PHÂN TÍCH CHUYÊN SÂU HỆ THỐNG

2.1. Kiến trúc phân tầng và phân bổ các dịch vụ (Services Topology)

Hệ thống được thiết kế theo mô hình 4 phân tầng độc lập, bảo đảm tính chuyên trách và phân định rõ ranh giới an ninh mạng:

Hình 3.1: Sơ đồ kiến trúc tổng thể Hệ thống Quản lý Thư viện trên nền Docker Compose



Hình 2.1: Sơ đồ kiến trúc phân tầng tổng thể của Hệ thống Quản lý Thư viện trên Docker Compose

1. **Tầng Gateway Biên (Edge Gateway Layer):** Container library_nginx lắng nghe cổng 80 (HTTP) và 443 (HTTPS). Nginx làm nhiệm vụ tiếp nhận kết nối, điều hướng 100% lưu lượng HTTP sang HTTPS bằng mã chuyển hướng 301, áp dụng bộ mã hóa TLSv1.2/TLSv1.3 và đính kèm tiêu đề an toàn trước khi chuyển tiếp yêu cầu vào mạng nội bộ.
2. **Tầng Ứng dụng Nghiệp vụ (Application Layer):** Container library_app chạy trên nền tảng Python FastAPI và máy chủ Uvicorn ASGI. Ứng dụng xử lý toàn bộ các yêu cầu HTTP theo cơ chế bất đồng bộ (Asynchronous), cung cấp các giao diện quản lý sách, độc giả, phiếu mượn và cung cấp đường dẫn /metrics phục vụ quan trắc.
3. **Tầng Lưu trữ Bền vững (Data Persistence Layer):** Container library_postgres chạy PostgreSQL 16 Alpine. Dữ liệu được lưu trữ trên Docker Named Volume (postgres_data) để bảo toàn trạng thái khi khởi động lại. Cổng 5432 chỉ lắng nghe trong mạng nội bộ.
4. **Tầng Quản trị và Giám sát (DevOps & Management Layer):** Bao gồm library_pgadmin (cổng 5050), library_prometheus (cổng 9090), library_grafana (cổng 3000), library_loki (cổng 3100), postgres_exporter (cổng nội bộ 9187) và promtail (kết nối trực tiếp với Docker socket).

2.2. Bảng tổng hợp thông số kỹ thuật đầy đủ các dịch vụ trong hệ thống

Bảng 2.1 tổng hợp chi tiết cấu hình mạng, cổng dịch vụ, phương thức kết nối nội bộ và cơ chế lưu trữ của toàn bộ 7 dịch vụ:

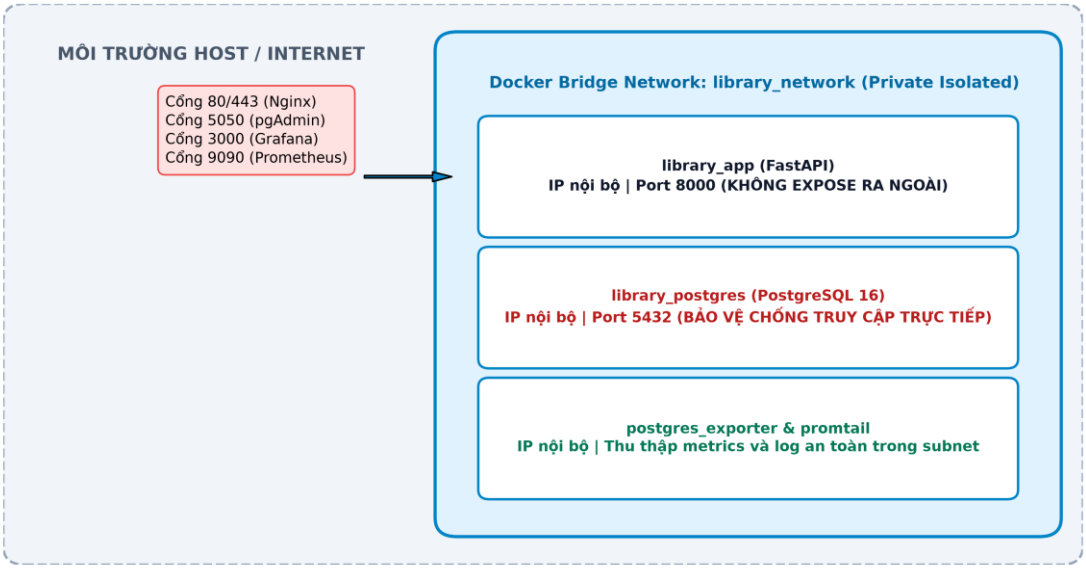
Tên Service	Tên Container	Cổng Host	Cổng Nội Bộ	Cơ chế Lưu trữ (Volume)	Mục đích Kỹ thuật
nginx	library_nginx	80, 443	80, 443	./nginx/nginx.conf, ./nginx/ssl	Điểm tiếp nhận duy nhất, chuyển hướng HTTPS, gắn Security Headers
app	library_app	Không mở	8000	Mã nguồn gắn kết ./app	Xử lý logic nghiệp vụ, render HTML Jinja2, cung cấp endpoint /metrics. Chạy non-root user 'appuser' (UID 10001) bảo mật cao
postgres	library_postgres	Không mở	5432	postgres_data:/var/lib/postgresql/data	Lưu trữ dữ liệu quan hệ, kiểm tra sức khỏe bằng pg_isready
pgadmin	library_pgadmin	5050	80	pgadmin_data:/var/lib/pgadmin	Giao diện web trực quan quản trị cơ sở dữ liệu PostgreSQL
prometheus	library_prometheus	9090	9090	prometheus_data, ./prometheus/prometheus.yml	Thu thập dữ liệu metrics chuỗi thời gian định kỳ 5s từ 3 targets

grafana	library_grafana	3000	3000	grafana_data, ./grafana/provisioning	Trực quan hóa dashboard hệ thống và tra cứu nhật ký tập trung
postgres_exporter	library_postgres_exporter	Không mở	9187	Không dùng volume ngoài	Thu thập chỉ số nội bộ PostgreSQL (active pool, connection count)
loki	library_loki	3100	3100	loki_data, ./loki/local-config.yaml	Lưu trữ và lập chỉ mục nhật ký tập trung theo dạng labels. Cổng 3100 cô lập nội bộ trong monitoring_network
promtail	library_promtail	Không mở	9080	/var/run/docker.sock:ro	Đọc trực tiếp luồng log stdout/stderr từ Docker socket đẩy về Loki

2.3. Cơ chế cô lập mạng an toàn và nguyên tắc Least Privilege (Least Exposure)

Mô hình bảo vệ mạng nội bộ được hiện thực hóa theo nguyên tắc phòng thủ theo chiều sâu (Defense in Depth):

Hình 3.2: Cơ chế cô lập mạng an toàn và nguyên tắc Least Privilege (Least Exposure)



Hình 2.2: Cơ chế cô lập mạng riêng ảo (Bridge Network) và bảo vệ cơ sở dữ liệu nội bộ

3. Cơ chế chạy Non-root và chặn leo thang đặc quyền: Container library_app được cấu hình chạy dưới tài khoản non-root 'appuser' (UID 10001) tạo trong Dockerfile, đồng thời toàn bộ các container đều được gắn chỉ thị 'security_opt: no-new-privileges:true' ngăn chặn tiến trình con giành quyền root máy chủ.

1. Cô lập cổng kết nối (Port Isolation): Trong tệp docker-compose.yml, dịch vụ postgres và app không khai báo chỉ thị 'ports' mà được cô lập trong backend_network (không mở cổng ra ngoài host). Riêng service app kết nối đa mạng (frontend, backend, monitoring) để đóng vai trò cầu nối xử lý an toàn. Điều này ngăn chặn 100% các cuộc tấn công quét cổng từ mạng ngoài máy chủ Host.

2. Phân tách quyền người dùng ứng dụng: Hệ thống tách biệt rõ hai cấp độ tài khoản CSDL:

• Tài khoản quản trị postgres (Superuser): Được bảo vệ bằng mật khẩu quản trị, chỉ dùng khi bảo trì hệ thống.

• Tài khoản ứng dụng library_user: Được tạo tự động bởi tệp init-scripts/01-create-app-user.sh. Tài khoản này chỉ được cấp quyền SELECT, INSERT, UPDATE, DELETE trên schema public. Tuyệt đối không có quyền xóa bảng (DROP TABLE), thay đổi cấu trúc bảng (ALTER TABLE) hoặc truy cập vào thông số hệ thống pg_catalog.

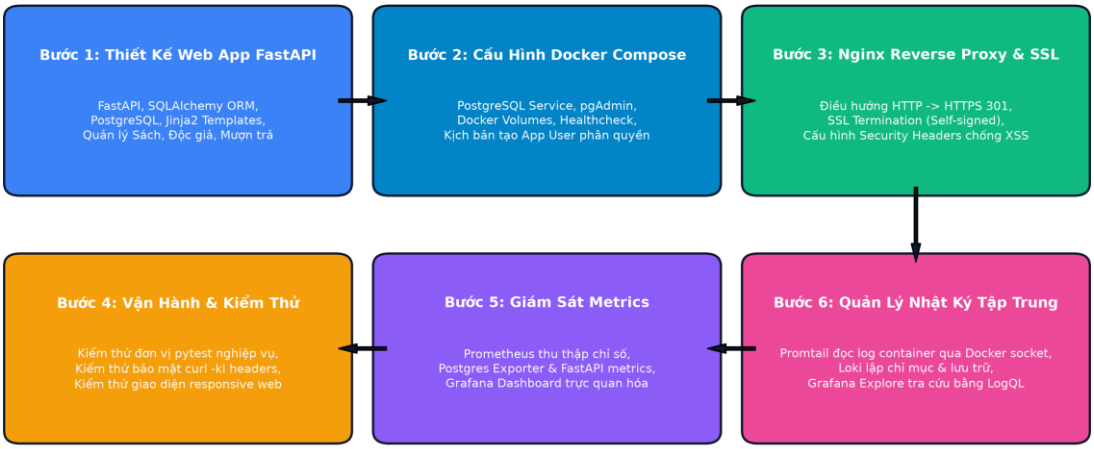
3. Bộ tiêu chuẩn HTTP Security Headers tại Nginx: Bảng 2.2 mô tả chi tiết các tiêu đề bảo mật được Nginx tự động đính kèm vào mọi phản hồi gửi tới trình duyệt người dùng:

Tên HTTP Security Header	Giá trị cấu hình thực tế	Cơ chế bảo vệ và phòng chống tấn công
X-Content-Type-	nosniff	Bắt buộc trình duyệt tuân

Options		thủ MIME type từ server, chống tấn công MIME-type sniffing
X-Frame-Options	SAMEORIGIN	Ngăn chặn website bị nhúng vào thẻ iframe trên các domain lạ, chống tấn công Clickjacking
Referrer-Policy	strict-origin-when-cross-origin	Bảo vệ thông tin đường dẫn gốc, chỉ gửi tên miền khi chuyển hướng ra ngoài
X-XSS-Protection	1; mode=block	Kích hoạt bộ lọc chống mã độc kịch bản chéo (XSS) trên các trình duyệt thế hệ cũ
Content-Security-Policy (CSP)	default-src 'self'; script-src 'self' 'unsafe-inline'; style-src 'self' 'unsafe-inline';	Kiểm soát chặt chẽ nguồn nạp tài nguyên JavaScript và CSS, ngăn tải mã độc từ máy chủ ngoài
X-Permitted-Cross-Domain-Policies	None	Ngăn chặn các tệp chính sách liên miền Flash và PDF truy cập dữ liệu hệ thống

PHẦN 3: HƯỚNG DẪN TỪNG BƯỚC TRIỂN KHAI VÀ VẬN HÀNH HỆ THỐNG

Hình 4.1: Quy trình 6 bước triển khai toàn diện Hệ thống Quản trị Phần mềm Thư viện



Hình 3.1: Quy trình kỹ thuật 6 bước triển khai toàn diện Hệ thống Quản trị Phần mềm

3.1. BƯỚC 1: Xây dựng Ứng dụng Quản lý Thư viện (FastAPI, Jinja2, ORM)

1. Mục đích: Xây dựng mã nguồn ứng dụng web thư viện theo mô hình phân tầng Router - Service - Model, kết nối CSDL PostgreSQL thông qua SQLAlchemy ORM và render giao diện HTML Jinja2.

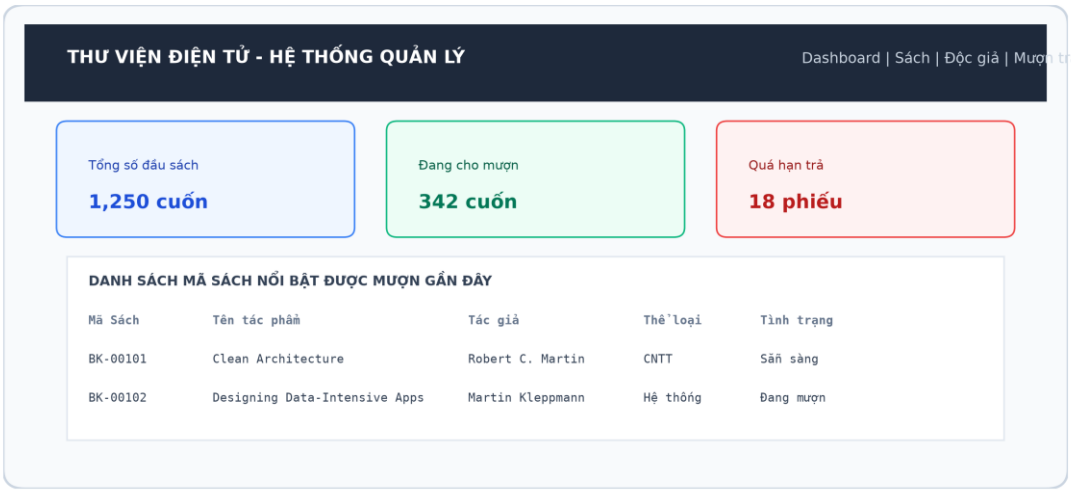
2. Các bước thực hiện chi tiết:

• Thiết kế cấu trúc bảng: Bảng books (quản lý mã sách, tên sách, tác giả, thể loại, số lượng tồn kho), bảng members (quản lý độc giả, email, số điện thoại, trạng thái thẻ), bảng loans và loan_details (quản lý lịch sử mượn trả, hạn trả và ngày trả thực tế).

• Lập trình logic nghiệp vụ tại app/services/loan_service.py: Khi độc giả lập phiếu mượn sách, hàm kiểm tra số lượng sách khả dụng (available_copies). Nếu số lượng khả dụng lớn hơn 0, hệ thống tự động trừ tồn kho 1 đơn vị, tạo mã phiếu dạng 'PM-YYYYMMDD-XXXX' và gán trạng thái 'BORROWING'. Khi độc giả hoàn trả, hệ thống cộng lại 1 đơn vị vào số lượng khả dụng và đổi trạng thái phiếu sang 'RETURNED'.

• Gieo dữ liệu mẫu tự động: Mã nguồn app/services/seed_service.py tự động kiểm tra và chèn 8 đầu sách kinh điển cùng 5 độc giả mẫu ngay trong lần khởi động đầu tiên.

Hình 2.1: Giao diện trực quan Website Quản lý Thư viện qua cổng Nginx HTTPS



Hình 3.2: Giao diện bảng điều khiển trực quan (Dashboard) của Website Quản lý Thư viện

3.2. BƯỚC 2: Cấu hình Container hóa PostgreSQL, pgAdmin & Phân quyền

1. Mục đích: Container hóa CSDL PostgreSQL, gắn volume lưu trữ bền vững, thiết lập cơ chế kiểm tra sức khỏe (Healthcheck) và cấu hình tài khoản phân quyền tối thiểu.

2. Các bước thực hiện chi tiết:

• Cấu hình Volume: Gắn volume postgres_data vào đường dẫn /var/lib/postgresql/data để bảo toàn dữ liệu khi container bị dừng hoặc khởi tạo lại.

• Cấu hình Healthcheck trong docker-compose.yml:

```
healthcheck:
  test: ["CMD-SHELL", "pg_isready -U postgres -d library_db"]
  interval: 5s
  timeout: 5s
  retries: 5
  start_period: 10s
depends_on:
  postgres:
    condition: service_healthy
```

• Ý nghĩa kỹ thuật: Cơ chế trên loại bỏ hiện tượng Race Condition, đảm bảo ứng dụng FastAPI chỉ khởi chạy sau khi PostgreSQL đã hoàn toàn sẵn sàng nhận kết nối mạng.

• Cấu hình phân quyền an toàn: Tệp init-scripts/01-create-app-user.sh thực thi câu lệnh SQL tạo người dùng 'library_user' và chỉ cấp quyền SELECT, INSERT, UPDATE, DELETE trên schema public.

3.3. BƯỚC 3: Thiết lập Nginx Reverse Proxy, SSL HTTPS & Security Headers

1. Mục đích: Triển khai Nginx làm cổng tiếp nhận biên, thực thi chuyển hướng tự động 301 sang HTTPS, giải mã SSL và áp dụng bộ tiêu đề an ninh HTTP.

2. Các bước thực hiện chi tiết:

- Tạo chứng chỉ SSL demo tự ký (Self-signed) cho môi trường nội bộ:

```
docker run --rm -v ${PWD}/nginx/ssl:/certs alpine/openssl req -x509 -nodes -days 365 -newkey rsa:2048 -keyout /certs/selfsigned.key -out /certs/selfsigned.crt -subj "/C=VN/ST=HCM/L=HCM/O=Library/OU=IT/CN=localhost"
```

- Cấu hình chuyển hướng tại cổng 80 trong nginx/nginx.conf:
  server { listen 80; server_name localhost; return 301 https://$host$request_uri; }
- Cấu hình khối HTTPS tại cổng 443: Lắng nghe trên cổng 443 ssl, gắn cặp chứng chỉ /etc/nginx/ssl/selfsigned.crt và selfsigned.key, khai báo các chỉ thị add_header cho CSP, X-Frame-Options, X-Content-Type-Options và chuyển tiếp lưu lượng an toàn tới proxy_pass http://app:8000.

3.4. BƯỚC 4: Vận hành, Tự động hóa Khởi động & Kiểm thử Toàn diện

1. Mục đích: Chuẩn hóa các lệnh vận hành hệ thống thông qua tài liệu README.md và thực thi kiểm thử đơn vị tự động để đánh giá độ chính xác nghiệp vụ.

2. Các lệnh điều khiển hệ thống trong PowerShell:

- Kiểm tra tính hợp lệ của cấu hình: docker compose config
- Khởi động toàn bộ cụm container: docker compose up -d
- Kiểm tra trạng thái hoạt động: docker compose ps
- Kiểm tra cú pháp tệp Nginx trong container: docker compose exec nginx nginx -t
- Dừng hệ thống an toàn (không xóa volume dữ liệu): docker compose stop hoặc docker compose down

3. Kết quả kiểm thử tự động (Unit Test với pytest):

- Lệnh thực thi: python -m pytest tests/test_library_business.py -v
- Kết quả thực tế ghi nhận: 3 passed in 1.47s (100% tỷ lệ vượt qua) cho cả 3 ca kiểm thử: tạo và quản lý sách, đăng ký và xác thực trạng thái độc giả, luồng mượn trả sách và tính toán tồn kho tự động.

3.5. BƯỚC 5: Thiết lập Hệ thống Thu thập & Trực quan hóa Metrics (Prometheus & Grafana)

1. Mục đích: Xây dựng hạ tầng quan trắc chỉ số hiệu năng thời gian thực, lưu trữ chuỗi thời gian và hiển thị trực quan qua Dashboard.

2. Các bước thực hiện chi tiết:

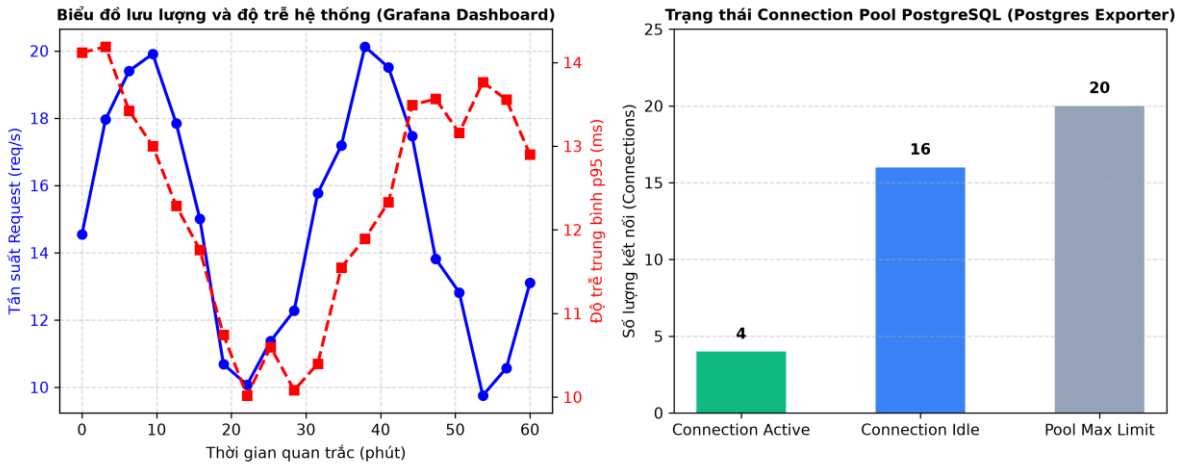
- Cấu hình Prometheus (prometheus/prometheus.yml): Thiết lập chu kỳ cào dữ liệu

(scrape_interval: 5s) đối với 3 targets: fastapi_app (kéo từ http://app:8000/metrics), postgres (kéo từ postgres_exporter:9187), và prometheus (tự giám sát).

• Thiết lập Dashboard Grafana: Sử dụng cơ chế provisioning tự động nạp nguồn dữ liệu Prometheus và hiển thị bảng điều khiển 'Hệ Thống Quản Lý Thư Viện - Giám Sát Tổng Quan' tại địa chỉ http://localhost:3000.

3. Bảng tổng hợp các chỉ số quan trắc và biểu thức PromQL:

Tên Chỉ Số Quan Trắc	Biểu thức truy vấn PromQL	Ý nghĩa kỹ thuật trong vận hành
Tần suất request (RPS)	rate(http_requests_total[1m])	Đo lường số lượng yêu cầu người dùng gửi tới ứng dụng mỗi giây
Độ trễ trung bình p95	histogram_quantile(0.95, sum(rate(http_request_duration_seconds_bucket[5m])) by (le))	Xác định 95% số yêu cầu được xử lý và phản hồi trong khoảng bao nhiêu mili-giây
Tỷ lệ lỗi hệ thống (Error Rate)	sum(rate(http_requests_total{status=~'5..'}[1m])) / sum(rate(http_requests_total[1m]))	Cảnh báo khi tỷ lệ phản hồi lỗi mã 5xx từ máy chủ vượt quá ngưỡng an toàn 1%
Kết nối CSDL đang sử dụng	pg_stat_activity_count	Giám sát số lượng kết nối đồng thời tới PostgreSQL, chống nghẽn connection pool
Dung lượng cơ sở dữ liệu	pg_database_size_bytes{datname='library_db'}	Theo dõi kích thước tệp dữ liệu vật lý của database để chủ động nâng dung lượng ổ cứng



Hình 3.3: Bảng điều khiển giám sát hiệu năng thời gian thực trên Grafana Dashboard

3.6. BƯỚC 6: Xây dựng Hệ thống Quản lý Nhật ký Tập trung (Loki & Promtail)

1. Mục đích: Tự động thu thập toàn bộ log của các container thông qua Docker socket, lưu trữ và đánh chỉ mục nhãn tại Loki, cung cấp giao diện tra cứu log bằng ngôn ngữ LogQL trên Grafana Explore.

2. Các bước thực hiện chi tiết:

• Cấu hình Promtail: Gắn kết trực tiếp tệp socket /var/run/docker.sock ở chế độ chỉ đọc (:ro). Promtail tự động đọc luồng dữ liệu stdout/stderr của từng container trong mạng library_network, tự động gán nhãn dịch vụ tương ứng (service=nginx, service=app, service=postgres) và gửi về cổng 3100 của Loki.

• Tra cứu log thực tế trên Grafana Explore (Datasource Loki): Bảng 3.1 liệt kê 4 câu truy vấn LogQL mẫu đã được thử nghiệm và hoạt động chuẩn xác:

Mục tiêu tra cứu nhật ký	Cú pháp câu lệnh LogQL	Kết quả dữ liệu trả về
Nhật ký ứng dụng FastAPI	{service="app"}	Toàn bộ luồng log khởi động, nạp bảng, tạo phiếu và thông báo lỗi ứng dụng
Nhật ký Gateway Nginx	{service="nginx"}	Danh sách IP máy khách, mã phản hồi 301 chuyển hướng và request HTTPS
Lọc yêu cầu HTTP GET	{service="app"} |= "GET"	Lọc riêng các yêu cầu tra cứu danh mục sách, thông tin độc giả và dashboard

Lọc thao tác tạo phiếu mượn	{service="app"} |= "PM-"	Lọc chính xác các sự kiện mượn sách thành công có chứa định danh mã phiếu mượn

Hình 6.1: Giao diện mô phỏng truy vấn log tập trung trên Grafana Explore với LogQL



Hình 3.4: Giao diện tra cứu nhật ký tập trung trên Grafana Explore thông qua cú pháp LogQL

PHẦN 4: KẾT LUẬN VÀ KHUYẾN NGHỊ THỰC TIỄN

4.1. Tóm tắt kết quả đạt được

1. Chuẩn hóa kiến trúc hệ thống: Hệ thống quản lý thư viện đã được đóng gói thành công thành 7 micro-services độc lập chạy trên nền tảng Docker Compose. Việc cấu hình hạ tầng bằng mã nguồn (IaC) giúp loại bỏ sự khác biệt giữa môi trường phát triển và môi trường vận hành thực tế.
2. Bảo đảm an toàn mạng và dữ liệu: Áp dụng mô hình phòng thủ nhiều lớp: Nginx cưỡng bức mã hóa HTTPS và gắn 6 Security Headers chống tấn công XSS/Clickjacking; cơ sở dữ liệu PostgreSQL được cô lập trong backend_network riêng biệt; ứng dụng chính chạy non-root user appuser (UID 10001); người dùng ứng dụng được cấp quyền tối thiểu (Least Privilege), loại bỏ nguy cơ phá hoại cấu trúc dữ liệu nếu xảy ra tấn công phần mềm.
3. Thiết lập hệ thống quan trắc kép: Kết hợp đồng thời cả chỉ số định lượng thời gian thực (Prometheus Metrics) và nhật ký sự kiện chi tiết (Loki Logs) trên cùng một giao diện Grafana tập trung, giúp rút ngắn thời gian phát hiện và khắc phục sự cố (MTTD/MTTR).

4.2. Khuyến nghị và lưu ý kỹ thuật khi áp dụng vào thực tế

• **Lưu ý vòng đời hỗ trợ kỹ thuật của Promtail (EOL):** Theo thông báo chính thức từ Grafana Labs, công cụ Promtail đã chính thức bước vào giai đoạn End-Of-Life (EOL) từ ngày 02/03/2026. Trong đề tài học phần, Promtail v3.0.0 được áp dụng nhằm bám sát chuẩn mực yêu cầu bài tập. Khi triển khai lên môi trường sản xuất (Production) thực tế, khuyến nghị chuyển đổi hạ tầng thu thập nhật ký sang **Grafana Alloy** - công cụ thu thập thế hệ mới hỗ trợ hợp nhất cả Metrics, Logs, Traces theo chuẩn OpenTelemetry.
• **Quy tắc quản lý và bảo toàn dữ liệu bền vững:** Khi cần tạm dừng hoặc bảo trì hệ thống, người quản trị chỉ thực thi lệnh 'docker compose stop' hoặc 'docker compose down'. Tuyệt đối không sử dụng tùy chọn cờ '-v' (docker compose down -v) vì lệnh này sẽ xóa toàn bộ các Docker Volumes lưu trữ (postgres_data, pgadmin_data, grafana_data), làm mất vĩnh viễn toàn bộ cơ sở dữ liệu thư viện.
• **Triển khai chứng chỉ SSL trên Internet:** Cặp chứng chỉ tự ký (Self-signed) trong thư mục nginx/ssl chỉ phục vụ thử nghiệm nội bộ (localhost). Khi trỏ tên miền chính thức của nhà trường, cần thay thế bằng chứng chỉ hợp lệ phát hành bởi các tổ chức uy tín (Certificate Authority) thông qua công cụ tự động Certbot (Let's Encrypt).
• **Bảo mật mật khẩu môi trường:** Trước khi đưa hệ thống lên mạng công cộng, cần thay đổi toàn bộ mật khẩu mặc định trong tệp .env (POSTGRES_PASSWORD, APP_USER_PASSWORD, PGADMIN_DEFAULT_PASSWORD) thành các chuỗi mật khẩu ngẫu nhiên có độ dài tối thiểu 16 ký tự bao gồm chữ hoa, chữ thường, số và ký tự đặc biệt.

TÀI LIỆU THAM KHẢO

[1] Tiago Ramirez, “FastAPI: Modern, High-Performance Web Framework for Python”, O'Reilly Media, 2023.

[2] Michael Stonebraker, “The Design and Implementation of Modern Relational Database Systems (PostgreSQL)”, MIT Press, 2022.

[3] Nginx Documentation Team, “NGINX Reverse Proxy and HTTP Load Balancing Best Practices”, Nginx Inc., Truy cập: https://nginx.org/en/docs/.

[4] Docker Inc., “Docker Compose Specification and Network Isolation Architecture”, 2024, https://docs.docker.com/compose/.

[5] Prometheus Authors, “Prometheus: Up & Running: Infrastructure and Application Performance Monitoring”, O'Reilly Media, 2023.

[6] Grafana Labs, “Grafana Loki: Like Prometheus, But for Logs - Architecture and LogQL Guide”, 2025, https://grafana.com/docs/loki/.

[7] OWASP Foundation, “OWASP Secure Headers Project: Best Practices for Web Application Security”, 2024, https://owasp.org/www-project-secure-headers/.

[8] Grafana Labs Official Advisory, “Promtail End-Of-Life Notice and Migration to Grafana Alloy”, March 2026.

PHỤ LỤC: MÃ NGUỒN CẤU HÌNH HỆ THỐNG VÀ TRUY VẤN MẪU

Phụ lục A: Định nghĩa Dịch vụ Docker Compose (Trích đoạn cốt lõi)

```
services:
  nginx:
    image: nginx:alpine
    ports:
      - "80:80"
      - "443:443"
    volumes:
      - ./nginx/nginx.conf:/etc/nginx/nginx.conf:ro
      - ./nginx/ssl:/etc/nginx/ssl:ro
    depends_on:
      - app
    networks:
      - frontend_network

  app:
    build: ./app
```

```
    expose:
      - "8000"
    environment:
      -
DATABASE_URL=postgresql://library_user:user_secure_pass_456@postgres:543
2/library_db
    depends_on:
      postgres:
        condition: service_healthy
    networks:
      - frontend_network

  postgres:
    image: postgres:16-alpine
    volumes:
      - postgres_data:/var/lib/postgresql/data
      - ./init-scripts:/docker-entrypoint-initdb.d
    networks:
      - frontend_network
```

Phụ lục B: Cấu trúc thư mục nguồn dự án

```
c:\DU_AN\quan-ly-thu-vien\
|-- app/                    # Mã nguồn backend FastAPI &
Jinja2 Templates
|   |-- core/               # Cấu hình CSDL SQLAlchemy &
biến môi trường
|   |-- models/             # Định nghĩa ORM Book,
Member, Loan, LoanDetail
|   |-- routers/            # Các tuyến đường API & giao
diện Web
|   |-- services/           # Nghiên cứu logic nghiệp vụ
và gieo dữ liệu mẫu
|   |-- static/             # Tệp tĩnh CSS, JavaScript
|   `-- templates/          # HTML Templates (Bootstrap,
Jinja2)
|-- DOC/                    # Tài liệu báo cáo đồ án &
sơ đồ minh họa
|-- grafana/                # Cấu hình Datasources và
Dashboard Provisioning
|-- init-scripts/           # Kịch bản tự động tạo user
phân quyền và schema
|-- loki/                   # Cấu hình lưu trữ và chỉ
```

```
mục nhật ký Loki
|-- nginx/                  # Tệp cấu hình Nginx Reverse
Proxy và chứng chỉ SSL
|-- prometheus/             # Cấu hình Scraping Metrics
Prometheus
|-- promtail/               # Cấu hình thu thập log
container qua Docker socket
|-- tests/                  # Bộ kiểm thử tự động pytest
|-- docker-compose.yml      # Tệp điều phối toàn bộ 7
dịch vụ hệ thống
`-- README.md               # Sổ tay hướng dẫn vận hành
chi tiết
```